# Acoular Workshop: Acoustic Data Acquisition and Processing in Python using a MEMS Microphone Array

Rabea Eschenhagen[1], Mikolaj Czuchaj[1], Gert Herold[1], Adam Kujawski[1], Oliver Lylloff[2], Art Pelling[1], Ennes Sarradj[1]

[1] *Fachgebiet Technische Akustik, TU Berlin, 10587 Berlin, Germany, Email: eschenhagen@campus.tu-berlin.de*

[2] *Department of Wind and Energy Systems, DTU, 4000 Roskilde, Denmark*

## Introduction

Microphone arrays are essential in applications like sound source localization, beamforming, and noise reduction. However, commercial microphone array hardware and software solutions are often expensive and out of reach for many researchers, engineers, and students interested in acoustics and IoT applications.

This contribution highlights using the open-source Acoular framework with an off-the-shelf 16-channel MEMS microphone array with an integrated USB audio interface as a cost-effective alternative for microphone array data acquisition and analysis. The capabilities of the Acoular framework in acquiring, streaming, and post-processing acoustic data are demonstrated with a focus on real-time applications. In addition, the SpectAcoular extension enables the integration of the Acoular framework with the visualization library Bokeh, creating interactive and operating system-independent client-server applications that can stream and analyze microphone array data in real time.

## Theory

In measurements with microphone arrays, each microphone $n$ records a superposition of sound pressure contributions from all active sources, which are assumed to be uncorrelated. The resulting pressure signal $p_n(t)$ at microphone $n$ can be expressed as follows:

$$p_n(t) = \sum_{m}^{M} a_{mn} q_m(t - \Delta\tau_{mn}). \tag{1}$$

$M$ denotes the number of active sources, $q_m(t)$ the source signal emitted by the source $m$, $a_{mn}$ an amplitude, and $\Delta\tau_{mn}$ the travel time from the source $m$ to the microphone $n$. Beamforming techniques exploit the known relative time delays between the microphones to focus on a specific direction or point in space. This enables the localization and visualization of the sources. In delay-and-sum beamforming, the time-delayed and amplitude-weighted signals from all microphones are summed:

$$p_m = \sum_{n}^{N} h_{mn} p_n(t + \Delta t_{mn}), \tag{2}$$

where $N$ is the number of microphones, $h_{mn}$ is an optional weight, and $\Delta t_{mn}$ is the time shift required to align signals for a chosen focus position. When this procedure is applied to a grid of focus points, the result is a map of beamformer output values that indicates potential source locations.

Because MEMS microphones typically provide a voltage proportional to the incident sound pressure, a calibration step is performed to convert the raw voltage $u(t)$ into physical units (pascals). If $\alpha$ the known sensitivity (V/Pa), then

$$p(t) = \frac{1}{\alpha} u(t). \tag{3}$$

## System setup

The system comprises hardware and software components as shown in Figure 1. The hardware includes a miniDSP UMA-16 v2 MEMS microphone array[1]. It features a uniform rectangular arrangement of 16 microphones and a central opening, where a standard USB camera can be integrated to enable synchronized video acquisition.

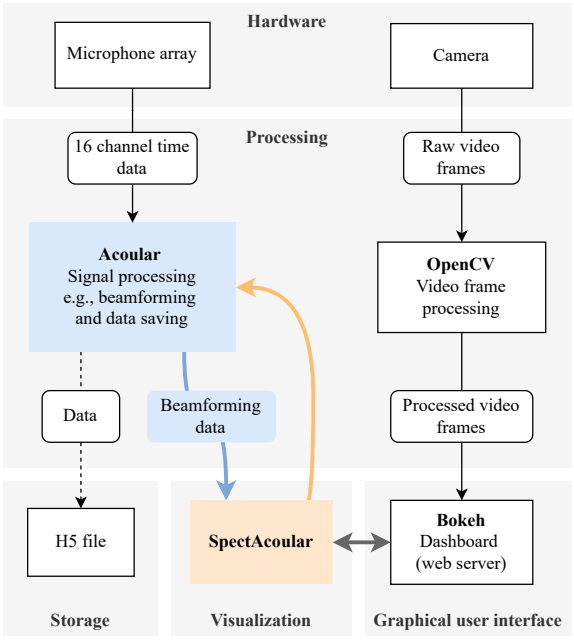



**Figure 1:** System overview: continuous real-time data acquisition, processing, and user interaction.

[1] https://www.minidsp.com/products/usb-audio-interface/uma-16-microphone-array (accessed: 14.11.2024)

The software components are implemented in Python and leverage several open-source libraries. The core functionality is provided by Acoular, a framework specialized in acoustic beamforming and multichannel microphone array data processing [1]. For real-time audio acquisition, Acoular integrates the sounddevice module, which provides access to audio hardware via the PortAudio backend and allows the configuration of input devices, sample rates, and buffer sizes [2, 3]. The Acoular extension SpectAcoular [4] provides interactive visualization and parameter control. SpectAcoular is based on the library Bokeh, a Python package for creating interactive web-based visualizations [5]. For real-time video capture and processing, SpectAcoular utilizes the OpenCV library, which provides tools for processing images and video streams [6].

## Data acquisition and processing

All implementation details and code examples discussed in this contribution are provided in an accompanying Jupyter notebook [7]. We encourage readers to run and modify the notebook to get hands-on experience with the methods.

The following example illustrates how microphone data can be acquired and processed using the Acoular framework. Listing 1 outlines the corresponding data acquisition procedure. Acoular enables a workflow that processes data blocks through a pipeline of Python generator objects [1]. Data acquisition begins with the `SoundDeviceSamplesGenerator`, which interfaces with the sound card to record multichannel audio data in real time. The acquisition process starts by specifying the device index and the number of channels. Since the raw microphone signals are measured in volts, they need to be converted into meaningful physical units, namely sound pressure in pascals (Pa). For this purpose, Acoular provides the `Calib` object, which scales the recorded signals according to the microphone sensitivity. In the example shown in Listing 1, a standard sensitivity of $0.016 V/Pa$ is assumed, resulting in a scaling factor of $1/0.016$. The calibrated data is then recorded for a predefined duration by specifying the number of samples to be acquired.

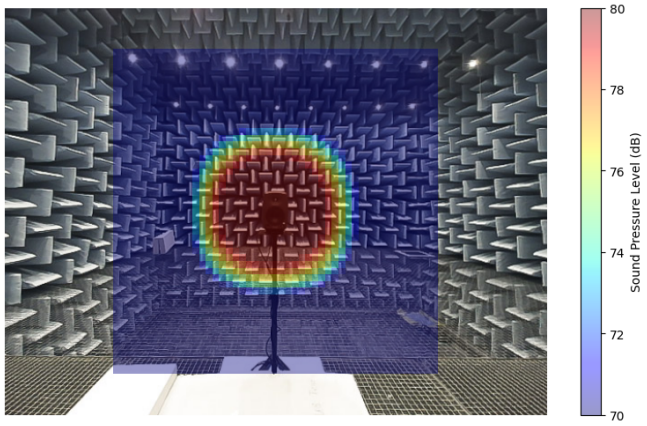



**Figure 2:** Beamforming result at 4 kHz overlaid on a photo of the measurement setup. The color map indicates sound pressure level (SPL) in dB.

Acoular's `tools.return_result()` function collects the data in blocks of a specified size and returns the complete signal as a NumPy array. Once the calibrated time domain data is available, it can be further processed or saved directly to disk. Acoular offers several data storage and processing classes, organized in the module `tprocess.py`. In the example shown, the class `WriteH5` saves the captured data in an HDF5 file format.

```
import acoular as ac
import numpy as np

dev = ac.SoundDeviceSamplesGenerator(
    device = device_number,
    numchannels = 16)

sf = 1/0.016 # scale factor
dev_c = ac.Calib(
    source = dev,
    data = sf * np.ones(dev.num_channels))

dev.num_samples = int(2 * dev.sample_freq)

signal = ac.tools.return_result(
    dev_c, num=256)

hdf5 = ac.WriteH5(
    source = dev_c,
    file = 'sounddevice.h5')

hdf5.save()
```

**Listing 1:** Setup of signal acquisition and conversion to sound pressure.

To illustrate the subsequent processing steps, we now apply a simple frequency-domain beamforming procedure to the recorded MEMS microphone data. After conversion to physical units, the signals are segmented and windowed using a Hanning window [1]. The resulting spectral data is computed using the cross-spectral matrix with Acoular's `PowerSpectra` class.

```
grid = ac.RectGrid(
    x_min=-1, x_max=1,
    y_min=-1, y_max=1,
    z=1.5, increment=0.05)

steer = ac.SteeringVector(
    mics=ac.MicGeom(file=path), grid=grid,
    env=ac.Environment(c=c))

freq_data = ac.PowerSpectra(
    source=dev, window='Hanning',
    overlap='50%', block_size=128)

bb = ac.BeamformerBase(
    freq_data=freq_data, steer=steer)

frequency = 4000
result = bb.synthetic(frequency, num=3)
Lm = ac.L_p(result)
Lm_reshaped = Lm.reshape(grid.shape)
```

**Listing 2:** Frequency-domain beamforming workflow using Acoular.

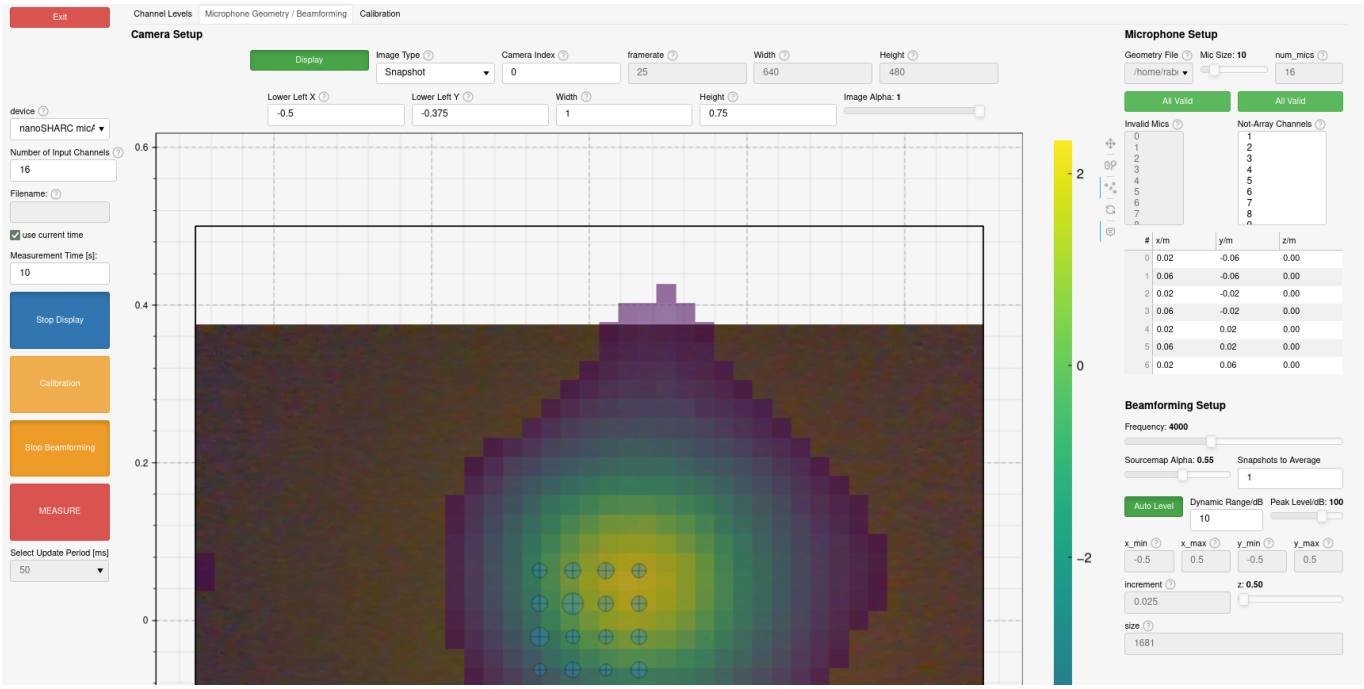



**Figure 3:** SpectAcoular's microphone array measurement app, an interactive application for sound source localization.

Listing 2 provides an example of this workflow. It defines the spatial evaluation grid and steering vector, then applies delay-and-sum beamforming at a center frequency of 4 kHz. The beamforming output is converted to decibel values and reshaped to match the grid layout, resulting in a spatial map highlighting the dominant sound sources. Figure 2 shows the beamforming result at 4 kHz. The calculated sound pressure levels are visualized as a heatmap and overlaid on a photograph of the measurement setup.

## Visualization with SpectAcoular

SpectAcoular is an extension module built on Acoular and the Bokeh visualization library. It provides a framework for creating custom web-based applications for interactive microphone array processing. SpectAcoular translates Acoular's class attributes (traits) into interactive widgets, allowing users to adjust parameters such as grid size, frequency, and dynamic range at runtime without changing the source code [4]. Applications created with SpectAcoular run in a web browser using a client-server architecture. The Bokeh server manages communication between Python and the browser interface, and synchronizes widget states and visualizations in real time. This configuration allows the system to be independent of the operating system, since the actual display and interaction remain on the client side via the BokehJS JavaScript API. An example of this widget integration is shown in Listing 3, where a `RectGrid` object is created and its interactive controls are displayed using Bokeh.

All the abovementioned components are combined in SpectAcoular's prebuilt *microphone array measurement app*[2] to enable real-time acquisition, visualization, and beamforming of multichannel audio data. The app supports input streams from devices such as the MEMS microphone array, displays channel levels, and provides beamforming functionality based on the Acoular processing chain. Through a graphical interface, users can select microphone geometries, adjust processing parameters, and control the data acquisition. The GUI provides, among other features, camera alignment and live beamforming visualization, as shown in Figure 3.

```
from spectacoular import RectGrid
from bokeh.io import show, output_notebook
from Bokeh.layouts import column

output_notebook()
grid = RectGrid()
widgets = grid.get_widgets()
widget_list = list(widgets.values())
col = column(widget_list)
show(col)
```

**Listing 3:** Example code for usage of the SpectAcoular extension.

## Summary

This contribution presented a signal processing approach based on the Acoular framework to realize a low-cost acoustic camera. The system enables real-time acoustic data acquisition, processing, and visualization with minimal implementation effort. Combining off-the-shelf MEMS microphone arrays and open-source software offers a cost-effective alternative to commercial systems. The integration of web-based visualization frameworks such as Bokeh provides an intuitive way to implement platform-independent and interactive acoustic data analysis.

[2] https://acoular.github.io/spectacoular/apps/micgeomexample.html (accessed: 07.04.2025)

## References

[1] Sarradj, E., Herold, G.: A Python framework for microphone array data processing. Applied Acoustics 116 (2017), 50–58, DOI: 10.1016/j.apacoust.2016.09.015

[2] Geier, M.: Sounddevice: Play and record sound with Python, 2020

[3] Sarradj, E., Herold, G., Kujawski, A., Jekosch, S., Pelling, A. J. R., Czuchaj, M., Gensch, T., Oertwig, S., Tschavoll, J., Ostendorf, J., Schumacher, T.: Acoular – Acoustic testing and source mapping software. Version v25.03, Zenodo, 2025, DOI: 10.5281/ZENODO.15019298

[4] Kujawski, A., Herold, G., Sarradj, E., Jekosch, S.: SpectAcoular! an Extension for Acoular. Fortschritte der Akustik - DAGA 2021, 47. Jahrestagung für Akustik, Wien, 824–827, Deutsche Gesellschaft für Akustik e.V. (DEGA), 2021

[5] Bokeh Development Team: Bokeh: Python library for interactive visualization, URL: `https://bokeh.org/`, accessed: 2024-11-14, 2018

[6] Bradski, G.: The OpenCV Library. Dr. Dobb's Journal of Software Tools, 2000

[7] Sarradj, E., Herold, G., Kujawski, A., Pelling, A. J. R., Czuchaj, M., Tschavoll, J., Lylloff, O., Eschenhagen, R.: Acoular workshop – DAS — DAGA 2025, Version v25.04, 2025, DOI: 10.5281/ZENODO.15112633